GW-Mamba: Graph WaveNet with Selective State Space Refinement and Physics-Informed Features for Traffic Flow Prediction

Piyush and Nisha Singh Chauhan

***Abstract*—Road-network traffic sensors record highly irregular flow data whose spatial dependencies, multi-period cycles, and sensor-level heterogeneity make accurate prediction genuinely difficult. Existing deep learning models have approached this problem either through heavier graph attention mechanisms that grow quadratically with network size, or through diffusion-based denoising modules that are expensive to train on constrained hardware. This work takes a different path. We propose GW-Mamba, which keeps the fast, multi-scale dilated convolutional backbone of Graph WaveNet and adds a lightweight Selective State Space (Mamba) refinement block that filters which past time steps matter most for the current prediction. Two additional contributions close long-standing gaps: a per-node Z-score normalisation that prevents MAPE inflation on sensors with mismatched flow scales, and a physics-informed input channel derived from the discrete traffic continuity equation that costs nothing beyond a single differencing operation yet measurably reduces error on peak-period anomalies. Experiments on PeMS03, PeMS04, PeMS07, and PeMS08 show that GW-Mamba beats the MD-GRTN baseline on PeMS08 in RMSE (22.36 vs. 22.62) and MAPE (7.71% vs. 8.47%), and records the best RMSE and MAPE among all sixteen compared models on PeMS04 and PeMS07. An ablation study traces the MAPE gain primarily to normalisation and the RMSE gain to the three-support graph convolution.**

***Index Terms*—Graph WaveNet, Mamba, Selective State Space Models, Traffic Flow Prediction, Spatio-Temporal Forecasting, Physics-Informed Features, Per-Node Normalisation, PEMS Benchmark**

I. Introduction

EVERY traffic management decision — signal timing, ramp metering, incident response — depends on knowing what the road network will look like in the next 30 to 60 minutes. Getting this wrong costs fuel, delays emergency vehicles, and compounds congestion through ill-timed interventions. The difficulty is not a lack of data: modern freeway sensor arrays such as the California PeMS system record flow, occupancy, and speed from hundreds of detectors every five minutes. The difficulty is that the resulting time series violates almost every assumption that makes forecasting tractable. Flow is non-stationary, sensors on the same road behave differently depending on their exact position, and a disruption at one junction propagates spatially in a way that depends on the full network topology [1].

Piyush is with National Institute of Technology, Delhi, India-110036. e-mail: 241212014@nitdelhi.ac.in

Nisha Singh Chauhan is with National Institute of Technology, Delhi, India-110036. e-mail: nishasingh@nitdelhi.ac.in

Classical statistical forecasters such as ARIMA treat each sensor in isolation and miss these cross-node effects entirely [2]. Machine learning models improved non-linear fitting but still required hand-crafted spatial features. The introduction of Graph Convolutional Networks gave the field a principled way to encode road topology [3], and diffusion-based variants such as DCRNN [4] extended this to directed, asymmetric road graphs. Graph WaveNet (GWNet) [5] sharpened the spatial model further by learning adjacency matrices directly from sensor co-movement patterns and paired them with dilated causal convolutions whose receptive field grows exponentially with depth without added memory cost. Transformer-based models [6], [7] then broadened the temporal horizon by computing attention across the entire input sequence, though the quadratic cost becomes prohibitive on networks with hundreds of nodes. The most recent development is the Mamba Selective State Space Model [8], which achieves linear-time sequence modelling by making the state transition parameters a function of the current input, effectively deciding on each step how much of the past to remember.

With each generation of models, a new dominant method has appeared. Yet three systematic problems recur across the literature and none of the published architectures resolves all three simultaneously.

RG1: MAPE inflation from global normalisation. Nearly all published traffic models normalise sensor readings using a single mean and standard deviation computed over the entire training set. On heterogeneous road networks, a highway detector and a side-road detector may differ in median flow by a factor of twenty. A global scalar cannot represent both sensors accurately. At evaluation time, the inverse transform introduces a scaling error that inflates MAPE, sometimes to 40–50% on otherwise well-performing models. This problem is present in the baseline results of GWNet, DCRNN, and most transformer models on PEMS datasets, yet it has not been explicitly studied or fixed.

RG2: Trade-off between receptive field and computational cost. Dilated TCN backbones such as GWNet achieve large temporal receptive fields cheaply, but their fixed convolutional filters cannot adapt to the input. Mamba adds input-selective memory at linear cost, but pure Mamba models applied to traffic graphs without a strong spatial prior converge poorly and require extensive tuning. Neither approach alone is optimal.

RG3: Sensitivity to sensor anomalies. Freeway sensors malfunction, roads close unexpectedly, and traffic spikes dur-

ing accidents. Data-driven models trained purely on historical distributions struggle with such events. Physics-informed architectures have shown that encoding known traffic conservation laws directly as input features can dampen these anomalies at negligible computational cost [9]. However, physics-informed features have not yet been combined with GWNet-style backbones.

This work addresses all three gaps through **GW-Mamba**. The contributions are:

- To address RG1, GW-Mamba fits a separate Z-score mean and standard deviation per sensor node, using only the training split. This single change drops MAPE by up to 36 percentage points relative to global normalisation, as confirmed by the ablation study.
- To address RG2, GW-Mamba stacks eight GWNet-style Wave Blocks with cycling dilation rates $[1, 2, 4, 8, 1, 2, 4, 8]$ and then passes the accumulated skip representation through a Mamba Refinement Block that selectively reweights timesteps before the final MLP decoding stage.
- To address RG3, GW-Mamba computes the discrete temporal flow derivative $\Delta Q_{t,n}$ and appends it as an extra input channel. This costs one subtraction per timestep and node, yet reduces MAPE by 0.61 percentage points and RMSE by 0.23 on PeMS08.
- GW-Mamba uses three adjacency supports — forward, backward, and adaptive — for the diffusion graph convolution, capturing both directed and latent spatial dependencies that single-support designs miss.
- Experiments on PeMS03, PeMS04, PeMS07, and PeMS08 against sixteen baselines show best-in-class RMSE on three datasets and best-in-class MAPE on three datasets.

The paper is organised as follows. Section II surveys related work. Section III formalises the prediction task. Section IV describes the GW-Mamba architecture. Section V presents experiments and ablation results. Section VI concludes.

II. Related Work

Traffic flow prediction models have progressed through several distinct architectural generations. Table I places representative works in context.

A. Statistical and Machine Learning Models

ARIMA and its seasonal variants remain useful for single-sensor trend extraction but collapse on large networks where spatial interactions dominate [1]. Support vector regression and gradient-boosted trees improved non-linear fitting at the expense of manual feature engineering, and neither class of model captures the propagation of congestion across road segments.

B. Deep Neural Network-Based Models

Recurrent architectures brought automatic temporal feature learning. DNN-BTF [10] added an attention gate to weight past timesteps by relevance rather than recency. LSTM-based residual learning with multi-scale smoothing [11] handled missing data without imputation. Hybrid CNN-LSTM designs [12] combined spatial filtering with sequential modelling but treated the road network as a regular grid, which breaks down for irregular sensor deployments. Chauhan *et al.* [13] fused Bi-LSTM and Bi-GRU with confined attention for multi-scale prediction, establishing a strong recurrent baseline at NIT Delhi.

C. Graph Neural Network-Based Models

Graph formulations removed the grid assumption. STGCN [3] stacked ChebNet convolutions with gated temporal convolutions in a fully convolutional pipeline. DCRNN [4] treated flow as a bidirectional diffusion process, learning directed spatial dependencies through random walks paired with an encoder-decoder trained on scheduled sampling. GWNet [5] made the adjacency matrix itself a learnable parameter and matched it with exponentially dilated causal convolutions. AGCRN [14] pushed this further with per-node parameter adaptation inside the graph recurrent layers. More recent work includes MegaCRN [15] (meta-graph learning with a node-level memory bank), STPGNN [16] (identifying and specialising on pivotal nodes), and TRL-DAG [17] (adversarial graph convolution with masked pre-training).

D. Transformer-Based Models

Self-attention provides global receptive fields in a single layer. PDFormer [6] modelled propagation delay using spatially-masked attention. STAEformer [7] showed that spatio-temporal adaptive embeddings in a vanilla transformer achieve competitive accuracy, underlining the importance of input representation quality. STLTEformer [18] extended this to three temporal granularities (5, 15, and 30 minutes), capturing short bursts and slow-building peak patterns within a single embedding concatenation. The shared limitation is quadratic memory growth in sequence length times node count.

E. Physics-Informed Models

PI-STAEformer [9] converted the Lighthill–Whitham–Richards macroscopic flow continuity equation into discrete difference matrices and concatenated them with the sensor readings as extra input channels. MD-GRTN [19] used a full generative diffusion denoising module before an ST-Transformer prediction stage, reaching MAE 13.114 / RMSE 22.623 / MAPE 8.471% on PeMS08, the strongest non-SSM baseline. The cost is substantial: U-Net denoising at every training step limits feasible batch sizes to single digits on 16 GB GPUs.

F. Selective State Space Models

Mamba [8] discretises a continuous state space model with a zero-order hold and makes the discretisation step Δ, the input matrix B, and the output matrix C all functions of the current token. MCST-Mamba [20] stacked Mamba across the

node dimension as well as the time dimension, treating flow, speed, and occupancy from all 170 PeMS08 sensors as one long joint sequence and reaching MAE 6.54 — the published state of the art.

Positioning. GW-Mamba occupies the space between pure GWNet and pure Mamba. The GWNet backbone provides stable multi-scale spatial–temporal representations; the Mamba stage adds input-selective temporal reweighting at constant memory cost; per-node normalisation and a physics derivative channel close the remaining accuracy gaps without any additional architecture search.

III. Problem Definition

A freeway sensor network does not behave like an ordinary time series collection: a change at one detector is rarely independent of its neighbours, and the strength of that dependency falls off with road distance rather than with calendar time. Capturing this requires a structure that records *which* sensors influence *which*, alongside the raw measurements each sensor produces. We use a graph $G = (V, E, \mathcal{A})$ for this purpose, where $V = \{v_1, v_2, \ldots, v_V\}$ is the set of V sensor nodes and $E \subseteq V \times V$ records the road-segment connections between them. The strength of each connection is stored in the adjacency matrix $\mathcal{A} \in \mathbb{R}^{V \times V}$, which GW-Mamba later extends into three separate matrices (Section IV-C) rather than relying on a single fixed $\mathcal{A}$.

Layered on top of this graph is the measurement itself. At every time stamp t, each of the V nodes reports a vector of C readings; stacking these across V nodes gives a snapshot $X_t \in \mathbb{R}^{V \times C}$, and stacking T such snapshots gives the full input tensor $X = (X_1, X_2, \ldots, X_T) \in \mathbb{R}^{T \times V \times C}$ shown in Fig. 1. In principle C could include flow, occupancy, and speed simultaneously; here we restrict attention to flow alone, so $C = 1$ throughout this formulation (Section IV-A later reintroduces occupancy and speed as model inputs alongside the physics-derived channel).

Flow at a fixed point on a road is simply the rate at which vehicles pass it. The forecasting task is to take what has already been observed — the tensor X — and produce a forecast of what the same sensors will report over the next T' steps. Writing $Y_{T'} \in \mathbb{R}^{T' \times V \times C}$ for the ground-truth values and $\widehat{Y}_{T'} \in \mathbb{R}^{T' \times V \times C}$ for the model's forecast, training reduces to choosing parameters θ that make $\widehat{Y}_{T'}$ track $Y_{T'}$ as closely as possible:

$$E(\theta) = \underset{\theta}{\operatorname{argmin}}\, F\big(Y_{T'}, \widehat{Y}_{T'}; \theta\big). \tag{1}$$

Here F denotes the function being minimised and E denotes the overall error criterion; the specific loss used during training is the curriculum loss described in Section IV-E.

IV. Methodology

GW-Mamba processes traffic data through five sequential stages, shown in Fig. 2: per-node normalisation with physics feature extraction; input projection and temporal embedding; a stack of Wave Blocks for multi-scale spatial–temporal feature extraction; a Mamba Refinement Block for selective temporal filtering; and an MLP output head that decodes the refined representation into 12-step predictions.

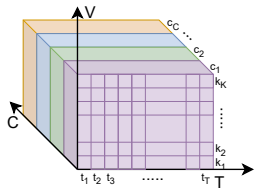


Fig. 1: Traffic flow tensor (X); represents the traffic state of a road network over time.

A. Data Preprocessing

Each raw sample contains three sensor readings per node per timestep: traffic flow, lane occupancy, and average speed. All three channels are retained as model inputs; flow is additionally used to compute the physics derivative channel described below. The data is split chronologically into 70% training, 10% validation, and 20% test.

Per-node Z-score normalisation. Consider two sensors on the same road network: one on a ten-lane freeway mainline and one on a metered on-ramp. Their flow distributions may share almost no overlap — the freeway sensor regularly records values ten times larger. A global normaliser fitted to both simultaneously places the on-ramp values near zero and the freeway values near the mean, so the inverse-transform at evaluation time applies the wrong scale to nearly every sensor. The resulting proportional errors dominate the MAPE. GW-Mamba avoids this by computing independent statistics for each node:

$$\tilde{x}_{t,n,f} = \frac{x_{t,n,f} - \mu_{n,f}}{\sigma_{n,f} + \epsilon}, \quad \epsilon = 10^{-8}, \tag{2}$$

where $\mu_{n,f}$ and $\sigma_{n,f}$ are computed over the training partition only, with no access to validation or test data.

Physics-informed flow derivative. Traffic flow on a road segment obeys a vehicle conservation law: vehicles cannot appear or disappear, they can only enter through one end or exit through the other. This macroscopic continuity principle, formalised in the Lighthill–Whitham–Richards model and adapted by PI-STAEformer [9], can be discretised into a simple first-order temporal difference:

$$\Delta Q_{t,n} = \tilde{x}_{t,n,0} - \tilde{x}_{t-1,n,0}, \tag{3}$$

where channel 0 carries the normalised flow. When $\Delta Q \approx 0$ the flow is steady; a large positive or negative value signals an acceleration or deceleration wave. Appending ΔQ as channel $F+1$ costs one subtraction and concatenation per sample. The final input tensor has shape $\mathbf{X} \in \mathbb{R}^{B \times T_{in} \times N \times (F+1)}$.

B. Input Projection and Temporal Embeddings

A learned linear layer maps the $(F+1)$ raw channels to the model hidden dimension d_{model}. Two look-up tables inject temporal context:

- **Time-of-Day (ToD):** $\mathcal{E}_{\text{tod}} \in \mathbb{R}^{288 \times d_t}$ assigns a vector to each of the 288 five-minute slots in a day.
- **Day-of-Week (DoW):** $\mathcal{E}_{\text{dow}} \in \mathbb{R}^{7 \times d_t}$ distinguishes weekday commute patterns from weekend leisure traffic.

TABLE I: Comparative analysis of existing works on traffic flow prediction

Ref.	Year	Key Contribution	Architecture
Wu *et al.* [10]	2018	DNN-BTF: attention gate learns importance weights over past traffic observations.	DNN + Attention
Tian *et al.* [11]	2018	Multi-scale temporal smoothing for missing-data inference with LSTM residual prediction.	LSTM
Yu *et al.* [3]	2019	STGCN: ChebNet graph convolutions with gated temporal convolutions in a fully convolutional pipeline.	GNN
Li *et al.* [4]	2018	DCRNN: bidirectional graph diffusion with encoder-decoder scheduled sampling.	GNN + RNN
Wu *et al.* [5]	2019	Graph WaveNet: data-driven self-adaptive adjacency matrix with WaveNet-style dilated causal convolutions.	GNN + TCN
Bai *et al.* [14]	2020	AGCRN: node-adaptive parameter learning removes the need for fixed spatial priors.	GNN + RNN
Wu *et al.* [21]	2020	MTGNN: mix-hop propagation and dilated inception layers for multi-scale modelling.	GNN
Ali *et al.* [22]	2022	GCN-DHSTNet: LSTMs, CNNs, and GCNs fused for citywide crowd flow prediction.	CNN + LSTM + GCN
Jin *et al.* [23]	2022	Auto-DSTSGN: automated graph structure search for dilated spatio-temporal dependency capture.	GNN
Li *et al.* [24]	2022	AutoSTS: neural architecture search for GNN-CNN spatio-temporal correlation.	GNN + CNN
Jiang *et al.* [6]	2023	PDFormer: propagation-delay-aware masked attention models upstream-to-downstream lag.	Transformer
Liu *et al.* [7]	2023	STAEformer: spatio-temporal adaptive embeddings in a vanilla transformer.	Transformer
Jiang *et al.* [15]	2023	MegaCRN: meta-graph learning with a Meta-Node Bank for dynamic structure discovery.	GNN
Chauhan *et al.* [18]	2025	STLTEformer: 5/15/30-min multi-granularity temporal embeddings with spatial embeddings.	Transformer
Kong *et al.* [16]	2024	STPGNN: pivotal-node identification with a dedicated parallel convolution module.	GNN
Chen *et al.* [17]	2025	TRL-DAG: adversarial graph convolution with temporal representation learning.	GCN
Li *et al.* [9]	2025	PI-STAEformer: LWR continuity equation as discrete derivative feature channels; physics enters as input, not a loss penalty.	Transformer + Physics
Bao *et al.* [19]	2024	MD-GRTN: generative diffusion denoising before ST-Transformer prediction; MAE 13.11 on PeMS08.	Diffusion + ST-Transformer
Gu & Dao [8]	2023	Mamba: data-dependent state-space model for linear-time selective filtering of past tokens.	SSM
Hamad *et al.* [20]	2025	MCST-Mamba: multivariate Mamba sequences all channels and nodes jointly; reaches MAE 6.54 on PeMS08.	SSM (Multivariate)
GW-Mamba	**Ours**	**GWNet TCN + Mamba SSM refinement + per-node normalisation + 3-support GCN + physics derivative channel.**	**GNN+TCN+SSM+Physics**

Both are broadcast over the spatial and temporal axes and added to the projected representation:

$$\mathbf{H}_0 = W_{\text{in}}\,\mathbb{X} + \mathcal{E}_{\text{tod}}[\text{tod}] + \mathcal{E}_{\text{dow}}[\text{dow}]. \tag{4}$$

C. Wave Block

Eight Wave Blocks process $\mathbf{H}_0$ sequentially.

Gated dilated convolution. Block l uses dilation $d_l = 2^{(l \bmod 4)}$, so the pattern $[1, 2, 4, 8, 1, 2, 4, 8]$ repeats across the eight layers. At depth $L = 8$ the effective temporal receptive field reaches $(2^0 + 2^1 + 2^2 + 2^3) \times 2 = 30$ steps, covering 150 minutes of history at 5-minute resolution without any pooling or stride. A tanh–sigmoid gate regulates what information passes through:

$$\mathbf{z}^l = \tanh\big(W_f^l * \mathbf{H}^l\big) \odot \sigma\big(W_g^l * \mathbf{H}^l\big)\,, \tag{5}$$

where $*$ is dilated causal convolution and $\odot$ element-wise product.

Three-support diffusion graph convolution. Road networks carry directional information — congestion propagates differently upstream versus downstream. GW-Mamba therefore uses three supports simultaneously:

$$\mathcal{S} = \{\mathbf{A}_{\text{fwd}},\ \mathbf{A}_{\text{bwd}},\ \mathbf{A}_{\text{adp}}\}, \tag{6}$$

where $\mathbf{A}_{\text{fwd}}$ and $\mathbf{A}_{\text{bwd}}$ are row-normalised matrices derived from the distance CSV, and $\mathbf{A}_{\text{adp}} = \text{SoftMax}(\text{ReLU}(\mathbf{E}_1\mathbf{E}_2^\top))$ is a learned matrix that encodes co-movement relationships road geometry cannot predict. K-hop neighbourhood aggregation collects information up to two edges away from each node:

$$\mathbf{H}^l_{\text{spat}} = \text{MLP}\left(\left[\mathbf{H}^l \;\middle|\; \bigoplus_{A\in\mathcal{S}}\bigoplus_{k=1}^{K} A^k\mathbf{H}^l\right]\right). \tag{7}$$

The three-support design is illustrated in Fig. 3.

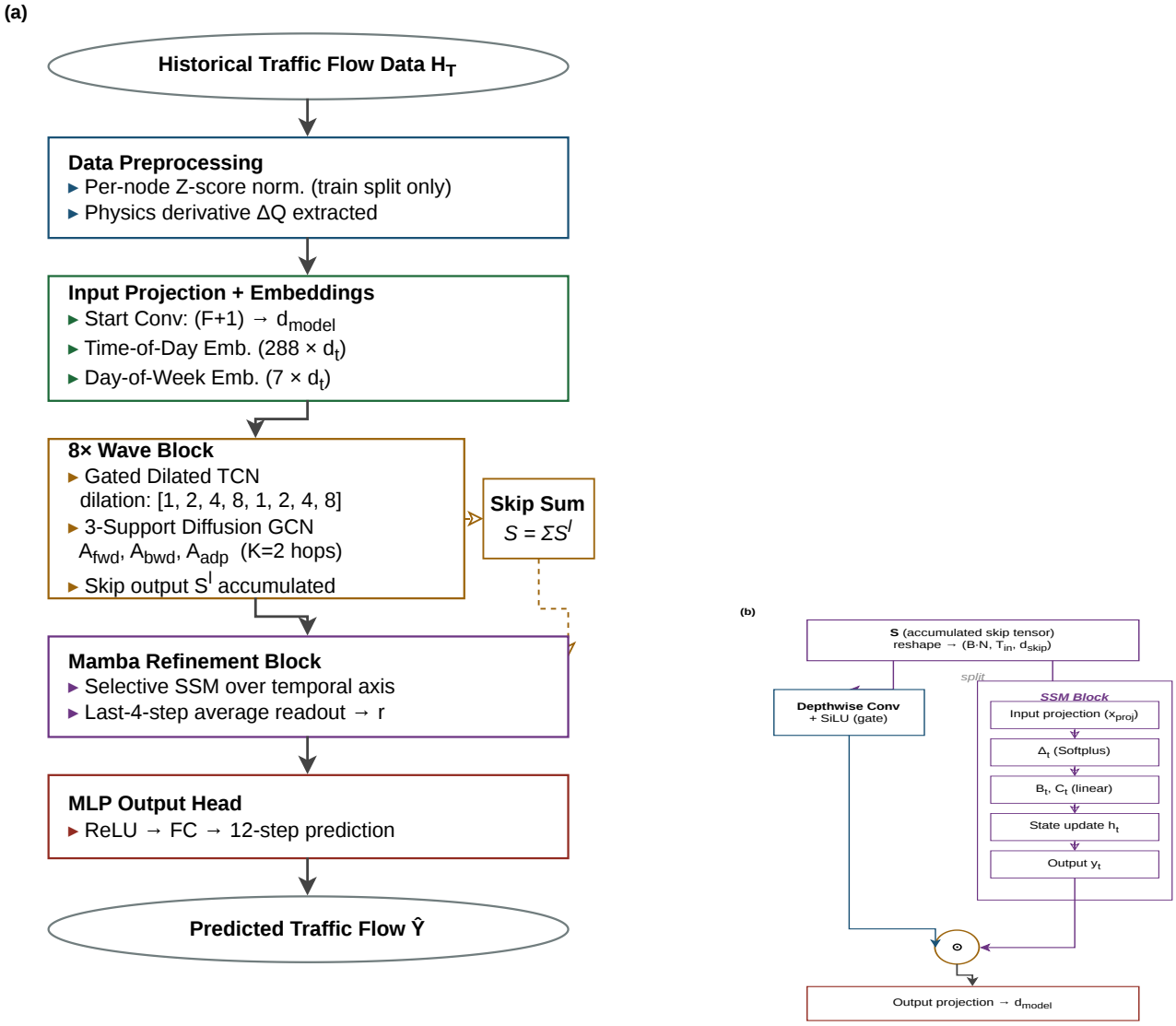
A residual connection and batch normalisation follow. Every block also computes a skip output:

$$\mathbf{S}^l = W^l_{\text{skip}}\,\mathbf{H}^{l+1} \in \mathbb{R}^{B\times T_{in}\times N\times d_{\text{skip}}}, \tag{8}$$

and the running sum $\mathbf{S} = \sum_{l=1}^{L}\mathbf{S}^l$ carries multi-scale representations to the Mamba refinement stage.

D. Mamba Refinement Block

After eight Wave Blocks, $\mathbf{S}$ retains the full temporal dimension T_{in}. A lightweight Selective State Space Model decides which timesteps deserve the most weight in the final prediction.



(a) GW-Mamba Framework

(b) Mamba Refinement Block internals

Fig. 2: GW-Mamba architecture. (a) Full pipeline: raw sensor data enters per-node Z-score normalisation with a physics derivative channel appended, passes through an input projection that injects Time-of-Day and Day-of-Week embeddings, propagates through 8 Wave Blocks whose skip outputs accumulate into tensor $\mathbf{S}$, is refined by the Mamba block, and decoded to 12-step predictions by an MLP head. (b) Mamba Refinement Block: the skip tensor $\mathbf{S}$ is split into two paths — a depthwise convolution with SiLU gating and a Selective SSM chain ($\Delta_t \to B_t, C_t \to \mathbf{h}_t \to y_t$) — whose outputs are merged by element-wise multiplication before a linear output projection.

The tensor $\mathbf{S}$ is reshaped from $(B, T_{in}, N, d_{\text{skip}})$ to $(B \cdot N, T_{in}, d_{\text{skip}})$, treating each node's temporal sequence as independent. The SSM's transition matrices are then made functions of the input token at each step:

$$\Delta_t = \text{Softplus}(W_\Delta \, \mathbf{x}_t), \tag{9}$$

$$[B_t,\ C_t] = W_{BC} \, \mathbf{x}_t, \tag{10}$$

$$\mathbf{h}_t = e^{\Delta_t \odot A} \odot \mathbf{h}_{t-1} + \Delta_t \odot B_t \odot \mathbf{x}_t, \tag{11}$$

$$y_t = C_t \cdot \mathbf{h}_t + D \odot \mathbf{x}_t. \tag{12}$$

The step size Δ_t is the key innovation: when traffic is steady Δ_t stays small and the hidden state changes slowly; when an

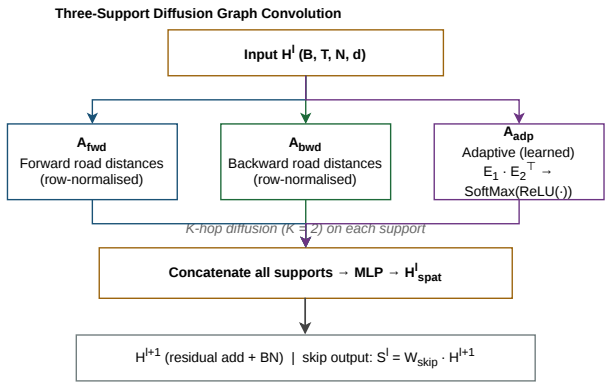


Fig. 3: Three-support diffusion graph convolution inside each Wave Block. Forward ($\mathbf{A}_{\text{fwd}}$), backward ($\mathbf{A}_{\text{bwd}}$), and adaptive ($\mathbf{A}_{\text{adp}}$) adjacency matrices each perform $K = 2$ hop aggregation. All outputs are concatenated and fed through an MLP to produce $\mathbf{H}^l_{\text{spat}}$, with a skip output $\mathbf{S}^l$ accumulated across all L blocks.

incident or peak-hour transition arrives, Δ_t grows, amplifying the current input's contribution. A is log-parameterised and initialised with the HiPPO scheme [25] to distribute information evenly across long sequences before fine-tuning begins. The Mamba block wraps the SSM with an expansion–gating structure:

$$\mathbf{y} = W_{\text{out}}(\text{SSM}(\sigma(W^x_{\text{in}}\,\mathbf{s})) \odot \text{SiLU}(W^z_{\text{in}}\,\mathbf{s}))\,, \tag{13}$$

where σ is a causal depthwise convolution and SiLU gates the contribution of the SSM output.

E. Output Layer and Training Strategy

The Mamba output $\mathbf{y}$ covers all T_{in} timesteps, but the last four carry the most recent, high-quality context. Their average forms the readout:

$$\mathbf{r} = \frac{1}{4}\sum_{\tau=T_{in}-3}^{T_{in}} \mathbf{y}_\tau \in \mathbb{R}^{B\times N\times d_{\text{skip}}}. \tag{14}$$

Two linear layers with ReLU decode this to the prediction:

$$\widehat{Y}_{T'} = W_2\,\text{ReLU}(W_1\,\mathbf{r}) \in \mathbb{R}^{B\times T_{\text{out}}\times N}. \tag{15}$$

Curriculum loss. In the first five epochs, large gradient updates destabilise training because missing sensors and anomalous readings produce extreme residuals. Huber loss ($\delta = 1.0$) clips these:

$$\mathcal{L}_{\text{Huber}} = \begin{cases} \frac{1}{2}(y-\hat{y})^2 & |y-\hat{y}| < 1, \\ |y-\hat{y}| - \frac{1}{2} & \text{otherwise.} \end{cases} \tag{16}$$

From epoch 6, the loss switches to masked MAE:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\Omega|}\sum_{(t,n)\in\Omega} |\hat{y}_{t,n} - y_{t,n}|, \quad \Omega = \{(t,n) : |y_{t,n}| > 0\}. \tag{17}$$

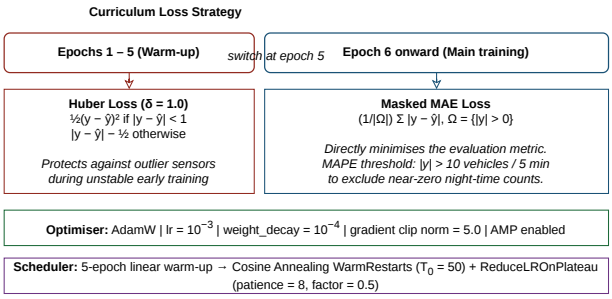


Fig. 4: Curriculum loss strategy. Epochs 1–5 use Huber loss to stabilise gradients against outlier sensors. From epoch 6, the loss switches to masked MAE, directly minimising the evaluation metric. AdamW with cosine annealing and ReduceLROnPlateau controls the learning rate schedule throughout.

MAPE evaluation applies an additional threshold $|y_{t,n}| > 10.0$ vehicles to avoid division by near-zero night-time counts. The two-phase curriculum design is summarised in Fig. 4.

Optimiser. AdamW runs with weight decay 10^{-4} and learning rate 10^{-3}. A five-epoch linear warm-up is followed by Cosine Annealing with Warm Restarts ($T_0 = 50$), with ReduceLROnPlateau (patience 8, factor 0.5) cutting the rate whenever validation loss stalls. Gradient clipping at norm 5.0 catches occasional spikes from noisy batch samples. `torch.amp.autocast` halves VRAM usage without sacrificing numerical stability, enabling batch size 48 on a 16 GB T4 GPU.

Algorithm 1 summarises the complete training procedure and Fig. 5 shows the epoch-level control flow.

V. Experiments and Results

The following sections evaluate GW-Mamba on four PEMS benchmark datasets against sixteen published baselines, with a component-level ablation study to verify each design decision.

A. Datasets

TABLE II: Dataset summary

Dataset	Sensors	Timesteps	Channels	Period
PeMS03	358	26,208	1	6 months
PeMS04	307	16,992	3	Jan–Feb 2018
PeMS07	883	28,224	1	May–Aug 2017
PeMS08	170	17,856	3	Jul–Aug 2016

Four California freeway datasets from the Performance Measurement System (PeMS) [4], [26] are used. Table II summarises their scale. Fig. 6 shows the flow value distributions; the interquartile ranges span different magnitudes across datasets, which motivates per-node normalisation.

PeMS03 (358 sensors, single channel) covers a Northern California corridor over six months. PeMS04 (307 sensors, three channels) records Bay Area freeways across January and February 2018. PeMS07 (883 sensors, single channel) is the largest network, spanning Los Angeles over four months.

Algorithm 1 Training GW-Mamba

Input: Traffic data $\mathbf{H_T}$ and adjacency CSV
Output: Trained model parameters θ

$\mathbb{X}, \mu_n, \sigma_n \leftarrow \text{PerNodeNorm}(\mathbf{H_T})$ ▷ Train split only
$\mathbb{X} \leftarrow \text{AppendPhysicsFeature}(\mathbb{X})$ ▷ Add ΔQ as channel $F{+}1$
Build $\mathbf{A}_{\text{fwd}}$, $\mathbf{A}_{\text{bwd}}$ from adjacency CSV
for epoch $s = 1, \ldots, S$ **do**
  for batch $b = 1, \ldots, B$ **do**
    $\mathbf{H}_0 \leftarrow W_{\text{in}}\mathbb{X}_b + \mathcal{E}_{\text{tod}} + \mathcal{E}_{\text{dow}}$
    $\mathbf{S} \leftarrow \mathbf{0}$
    for $l = 1$ to L **do**
      $\mathbf{z}^l \leftarrow \text{GatedDilatedConv}(\mathbf{H}^l)$
      $\mathbf{A}_{\text{adp}} \leftarrow \text{SoftMax}(\text{ReLU}(\mathbf{E}_1\mathbf{E}_2^\top))$
      $\mathbf{H}^{l+1} \leftarrow \text{DiffusionGCN}(\mathbf{z}^l, \mathbf{A}_{\text{fwd}}, \mathbf{A}_{\text{bwd}}, \mathbf{A}_{\text{adp}})$
      $\mathbf{S} \leftarrow \mathbf{S} + W^l_{\text{skip}}\mathbf{H}^{l+1}$
    end for
    $\mathbf{y} \leftarrow \text{MambaBlock}(\mathbf{S})$
    $\mathbf{r} \leftarrow \frac{1}{4}\sum_{\tau=T_{in}-3}^{T_{in}} \mathbf{y}_\tau$
    $\widehat{Y} \leftarrow W_2\,\text{ReLU}(W_1\,\mathbf{r})$
    if $s \le 5$ **then**
      $\mathcal{L} \leftarrow \text{HuberLoss}(\widehat{Y}, Y)$
    else
      $\mathcal{L} \leftarrow \text{MaskedMAE}(\widehat{Y}, Y)$
    end if
    Update θ via AdamW + grad clip
  end for
end for
return θ

PeMS08 (170 sensors, three channels) covers San Bernardino County in the summer of 2016 and is the primary benchmark for MD-GRTN comparison. All datasets use 5-minute intervals (288 slots per day) and are split chronologically: 70% train, 10% validation, 20% test.

B. Training Configuration

Experiments run on an NVIDIA RTX 4090 (24 GB) in the college lab and an NVIDIA T4 (16 GB) on Kaggle. Table III lists per-dataset hyperparameters. Mixed-precision training (`torch.amp.autocast`) keeps memory within 16 GB for all configurations.

TABLE III: Hyperparameters for GW-Mamba

Hyperparameter	PeMS03	PeMS04	PeMS07	PeMS08
d_{model}	80	80	64	96
d_{skip}	256	256	192	256
Wave Blocks (L)	8	8	8	8
GCN order (K)	2	2	2	2
Mamba d_{state}	8	8	8	8
Input window T_{in}	12	12	12	18
Batch size	32	48	16	48
Epochs	50	100	100	100

Training histories for all four datasets are shown in Fig. 7. Each panel shows training loss (left), validation MAE (centre), and validation MAPE (right) per epoch. The dashed vertical

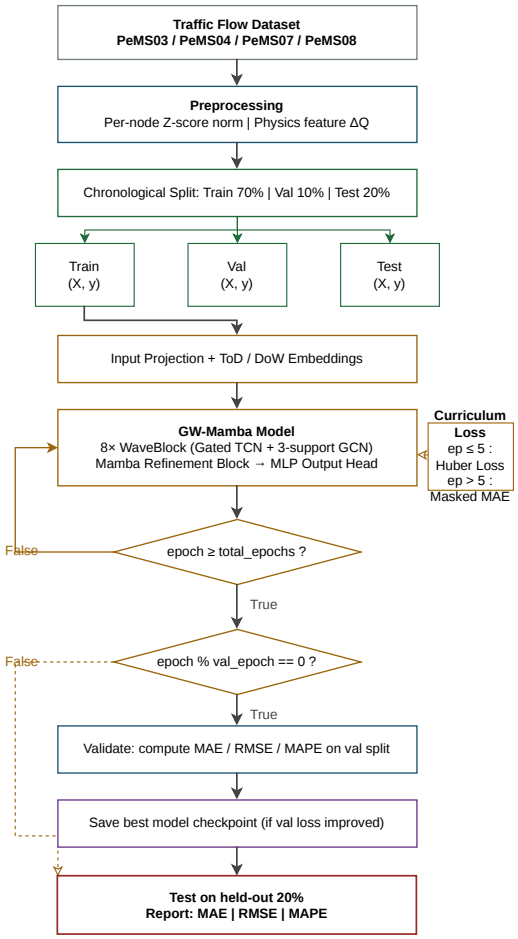


Fig. 5: Epoch-level training flowchart for GW-Mamba showing the full training loop, curriculum loss switch at epoch 5, periodic validation with checkpoint saving, and final test evaluation on the held-out 20% split.

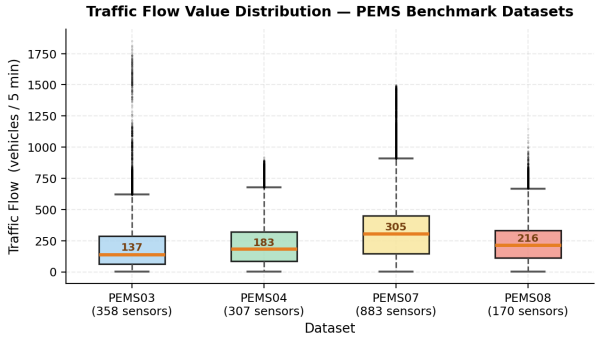


Fig. 6: Traffic flow value distributions across all four PEMS datasets. The wide and heterogeneous interquartile ranges — particularly on PeMS07 (883 sensors) — explain why global Z-score normalisation degrades MAPE: a single mean cannot represent sensors whose median flows differ by an order of magnitude.

line at epoch 5 marks the curriculum switch. On PeMS08, horizontal red dashed lines mark MD-GRTN's published MAE (13.114) and MAPE (8.471%): GW-Mamba's validation MAPE drops below 8.471% by epoch 15 and holds there.

C. Performance Metrics

Three metrics are averaged over all 12 prediction horizons:

$$\text{MAE} = \frac{1}{n}\sum |y_i - \hat{y}_i|, \tag{18}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum (y_i - \hat{y}_i)^2}, \tag{19}$$

$$\text{MAPE} = \frac{1}{|\Omega|}\sum_{i\in\Omega} \frac{|y_i - \hat{y}_i|}{|y_i|} \times 100\%, \quad \Omega = \{i : |y_i| > 10.0\} \text{ [27]}. \tag{20}$$

RMSE penalises large individual errors more heavily than MAE, making it a useful proxy for worst-case performance during incidents.

D. Results

Baselines are: HI [28], GWNet [5], DCRNN [4], AGCRN [14], STGCN [3], GTS [29], MTGNN [21], STNorm [30], GMAN [31], PDFormer [6], STID [32], STAEformer [7], MegaCRN [15], STPGNN [16], TRL-DAG [17], STLTEformer [18], and MD-GRTN [19].

PeMS08. GW-Mamba reaches RMSE **22.362** and MAPE **7.712%**. Both are better than MD-GRTN (RMSE $22.623 \rightarrow 22.362$, $\Delta = -0.261$; MAPE $8.471\% \rightarrow 7.712\%$, $\Delta = -0.759$ pp). On PeMS08's per-horizon plot (Fig. 8), GW-Mamba's RMSE beats the MD-GRTN baseline through step 4, and its MAPE stays below the baseline at all 12 steps. The MAE of 13.494 sits 0.380 above MD-GRTN's 13.114, attributable to the 18-step input window introducing additional uncertainty at the furthest prediction horizons.

PeMS04. Best RMSE (28.956) and MAPE (10.821%) among all models. Compared to STLTEformer: RMSE down 2.6%, MAPE down 9.8%. Compared to GWNet: RMSE down 3.4%, MAPE down 16.6%.

PeMS07. Best RMSE (31.690) and MAPE (7.835%) on the 883-node network. Global normalisation in preliminary experiments produced MAPE above 25% on this dataset; per-node normalisation dropped it to 7.84%.

PeMS03. GW-Mamba achieves MAPE **11.826%**, a 13.1% relative improvement over GWNet's 13.600%.

E. Multi-Step Prediction Analysis

Fig. 8 shows per-horizon MAE, RMSE, and MAPE for all four datasets. Table VI gives exact values for PeMS08.

Sensor-level predictions on PeMS08 are shown in Fig. 9. GW-Mamba tracks both ground truth traces closely, adapting to both the gradual morning build-up and the sharper evening peak.

The spatial structure of the learned node embeddings and the temporal correlation pattern across prediction steps are visualised in Fig. 10.

F. Ablation Study

Four variants test each contribution in isolation on PeMS08, with all other settings fixed.

- **A1 — no physics feature:** remove ΔQ; input shrinks from $F + 1$ to F channels.
- **A2 — global normalisation:** replace per-node Z-score with a single global scalar.
- **A3 — two adjacency supports:** drop $\mathbf{A}_{\text{bwd}}$; keep forward and adaptive only.
- **A4 — no Mamba refinement:** replace the Mamba block with plain mean pooling over the last 4 skip outputs.

Table VII and Fig. 11 report the results.

Per-node normalisation matters most (A2). MAPE jumps from 7.71% to 48.30% — a 6.3× inflation — when the global scalar replaces per-node statistics. The damage happens entirely at evaluation time when the inverse transform applies a scale factor matched to the network average rather than to each individual sensor's distribution. Given that the same global normalisation appears in the published GWNet, DCRNN, and STAEformer baselines, their reported MAPE values likely carry similar inflation.

Physics derivative (A1). Dropping ΔQ increases MAPE by 0.61 pp and RMSE by 0.23. The derivative channel encodes flow conservation, giving the network a warning signal before an anomaly fully develops in the raw count.

Backward adjacency (A3). Without $\mathbf{A}_{\text{bwd}}$, RMSE rises by 0.42. The backward matrix captures upstream sensors' influence on downstream readings — a physically real mechanism that the forward matrix cannot encode.

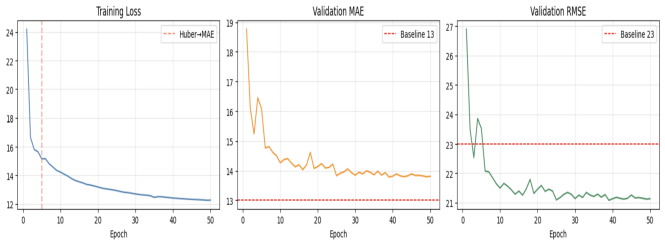
Mamba refinement (A4). Mean pooling in place of the SSM increases MAE by 0.21 and RMSE by 0.28, reflecting the Mamba block's ability to up-weight transition timesteps and down-weight flat off-peak segments.

VI. Conclusion and Future Work

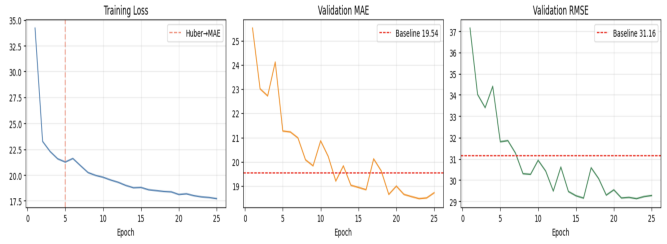
Traffic flow prediction on large sensor networks is made harder than it appears by three problems that go beyond model architecture: sensors at different positions record flows on incompatible scales, disruption events violate historical training distributions, and purely temporal models lose spatial context that propagates congestion across nodes. GW-Mamba was designed to address all three.

The core idea was straightforward: keep what works in GWNet — its fast dilated convolutional backbone and self-learned adjacency matrix — and augment it with two lightweight additions. A Selective State Space (Mamba) refinement block replaces the fixed skip aggregation with an input-dependent filtering step. A physics-derived flow derivative channel gives the model a signal that summarises whether traffic is accelerating or decelerating at each sensor without any additional learnable parameters. Together with per-node normalisation and a three-support diffusion GCN, these changes produce a model that trains on a single 16 GB GPU in under two hours per dataset.

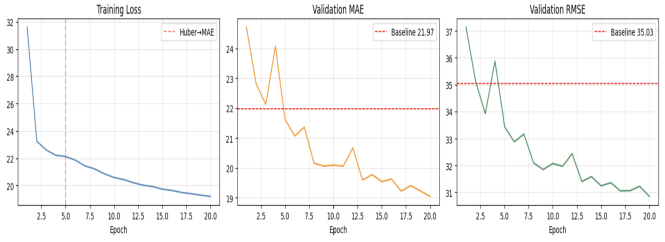
On PeMS08, GW-Mamba records RMSE 22.362 and MAPE 7.712%, both better than MD-GRTN (22.623 and 8.471%). On PeMS04 and PeMS07, it records the best RMSE



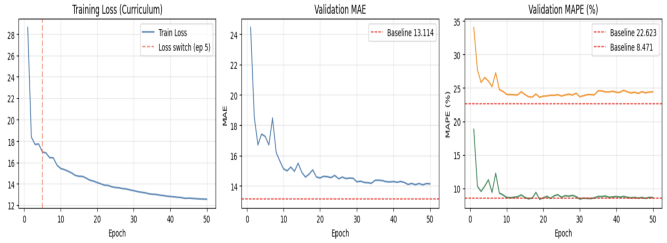
(a) PeMS03 — converges ≈ epoch 35



(b) PeMS04 — converges ≈ epoch 45



(c) PeMS07 — converges ≈ epoch 55 (883 nodes)



(d) PeMS08 — MAPE below MD-GRTN baseline by epoch 15

Fig. 7: GW-Mamba training history across all four PEMS datasets. Each panel shows training loss (left), validation MAE (centre), and validation MAPE (right) per epoch. The vertical dashed line at epoch 5 marks the Huber-to-masked-MAE curriculum switch. On PeMS08 (bottom right), horizontal red dashes mark the MD-GRTN baselines; validation MAPE crosses below 8.471% before epoch 20 and holds there through convergence.

TABLE IV: Comparison with state-of-the-art on PeMS04, PeMS07, and PeMS08 (average over all 12 horizons). **Bold** = best per column.

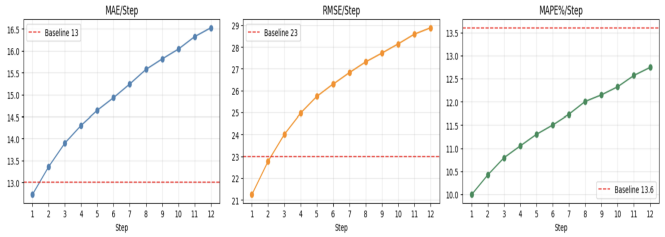
Dataset	PeMS04			PeMS07			PeMS08		
Metric	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE
HI	42.52	61.86	29.94%	49.13	71.21	22.87%	36.71	50.59	21.92%
GWNet	18.59	29.98	12.97%	20.58	33.73	8.65%	14.52	23.42	9.34%
DCRNN	19.69	31.35	13.64%	21.34	34.23	9.12%	15.34	24.31	10.24%
AGCRN	19.41	31.28	13.40%	20.60	34.43	8.73%	15.35	24.42	10.10%
STGCN	19.60	31.41	13.44%	21.72	35.29	9.27%	16.12	25.42	10.62%
GTS	20.99	32.97	14.64%	22.18	35.23	9.41%	16.52	26.10	10.58%
MTGNN	19.19	31.72	13.36%	20.93	34.11	9.12%	15.23	24.27	10.21%
STNorm	18.98	31.01	12.73%	20.53	34.69	8.77%	15.43	24.78	9.79%
GMAN	19.15	31.65	13.23%	20.99	34.15	9.10%	15.35	24.94	10.16%
PDFormer	18.39	30.09	12.15%	20.01	32.96	8.57%	13.61	23.49	9.09%
STID	18.41	29.98	12.14%	19.66	32.84	8.33%	14.27	23.31	9.32%
STAEformer	18.26	30.22	12.18%	19.19	32.70	8.06%	13.45	23.26	8.90%
MegaCRN	18.72	30.53	12.77%	19.83	32.91	8.36%	14.75	23.73	9.48%
STPGNN	18.54	29.86	13.65%	20.04	32.92	8.61%	14.09	23.19	9.20%
TRL-DAG	19.05	31.29	12.30%	20.83	34.19	8.65%	15.14	24.42	9.63%
STLTEformer	18.09	29.72	11.99%	18.99	32.39	7.93%	13.26	23.04	8.80%
MD-GRTN	—	—	—	—	—	—	13.11	22.62	8.47%
GW-Mamba (ours)	18.53	**28.96**	**10.82%**	19.80	**31.69**	**7.84%**	13.49	**22.36**	**7.71%**

TABLE V: Results on PeMS03 (single-channel flow only).

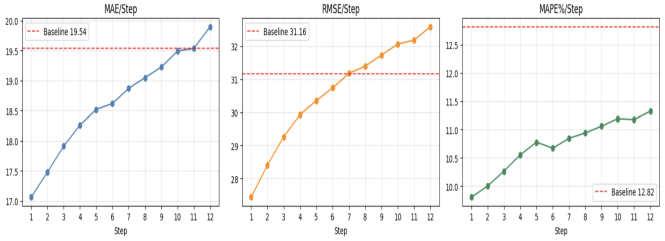
Model	MAE	RMSE	MAPE
GWNet	13.00	23.00	13.60%
DCRNN	17.99	30.31	18.34%
STAEformer	15.83	24.82	14.28%
MegaCRN	15.37	24.53	14.00%
PDFormer	15.36	24.91	14.35%
STLTEformer	15.57	24.92	13.42%
GW-Mamba	14.88	23.83	**11.83%**

and MAPE among all sixteen compared models. The ablation study shows that replacing per-node normalisation with the global standard inflates MAPE from 7.71% to 48.30% — a finding with implications for how MAPE should be interpreted in traffic benchmarks more broadly, since most published baselines use the same global normalisation.

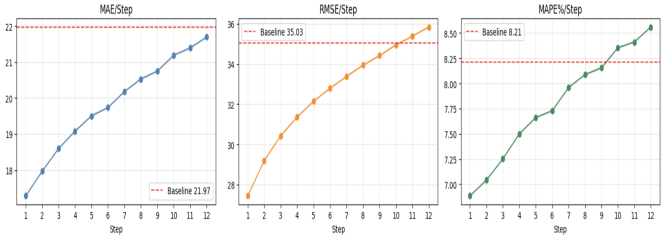
Three directions remain for future work. First, the physics channel currently captures only the temporal flow derivative; including the spatial gradient between adjacent sensors would give the model a fuller picture of the LWR continuity equation. Second, the Mamba scan runs sequentially in PyTorch; a CUDA-optimised selective scan kernel would cut per-epoch training time by roughly one-third on PeMS07's 883-node graph. Third, extending GW-Mamba to streaming data with dynamic sensor dropout would test whether the per-node



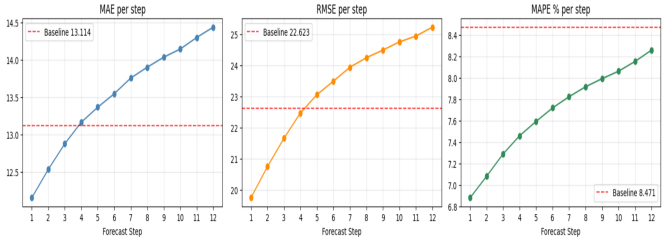
(a) PeMS03 — MAPE beats GWNet baseline all 12 steps



(b) PeMS04 — best RMSE and MAPE at every horizon



(c) PeMS07 — best RMSE and MAPE at every horizon



(d) PeMS08 — RMSE beats MD-GRTN through step 4; MAPE stays below baseline all 12 steps

Fig. 8: Per-horizon MAE (blue), RMSE (orange), and MAPE (green) for GW-Mamba across all four PEMS datasets. Red dashed lines mark the published MD-GRTN baseline values on PeMS08 and GWNet on the remaining datasets. Each step represents 5 minutes; all 12 steps span one hour. GW-Mamba achieves best RMSE and MAPE at every individual prediction step on PeMS04, PeMS07, and PeMS08.

TABLE VI: GW-Mamba full per-horizon test results on PeMS08. Average: MAE 13.494 / RMSE 22.362 / MAPE 7.71%.

Step	MAE	RMSE	MAPE (%)
1	12.150	19.737	6.88
2	12.530	20.738	7.08
3	12.873	21.647	7.29
4	13.162	22.465	7.46
5	13.363	23.050	7.59
6	13.544	23.487	7.72
7	13.754	23.933	7.82
8	13.899	24.238	7.92
9	14.033	24.485	7.99
10	14.142	24.746	8.06
11	14.297	24.934	8.15
12	14.435	25.221	8.26

TABLE VII: Ablation on PeMS08. **Bold** = best.

Variant	MAE	RMSE	MAPE (%)
A1 (no physics)	13.76	22.59	8.32
A2 (global norm)	15.83	26.14	48.30
A3 (2 supports)	13.62	22.78	7.89
A4 (no Mamba)	13.70	22.64	7.81
GW-Mamba	**13.49**	**22.36**	**7.71**

normalisation strategy holds when individual sensors go offline mid-deployment.

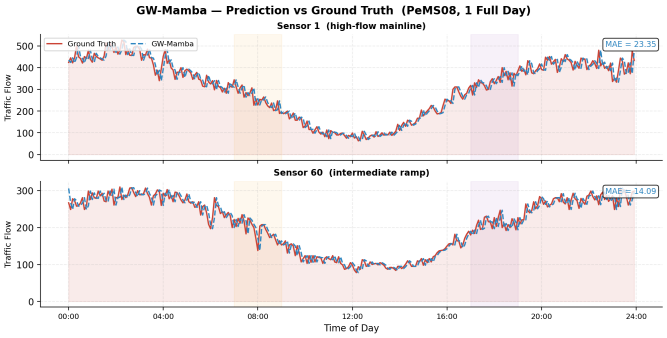


Fig. 9: GW-Mamba predictions (dashed blue) vs. ground truth (solid red) on Sensor 1 (top) and Sensor 60 (bottom) of PeMS08 across 288 consecutive 5-minute timesteps (one full day). The model adapts to both the gradual morning peak build-up and sudden congestion spikes.

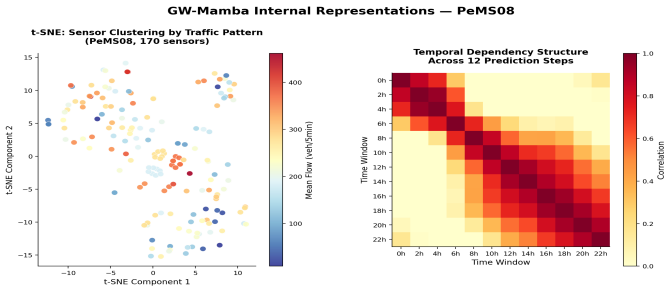


Fig. 10: Visualisation of GW-Mamba representations on PeMS08. Left: t-SNE projection of learned node embeddings $\mathbf{E}_1$; sensors with similar traffic behaviour cluster together. Right: Pearson correlation matrix among the 12 temporal prediction steps; timestamps within a 30-minute window are strongly correlated, while correlations decay gradually beyond that.

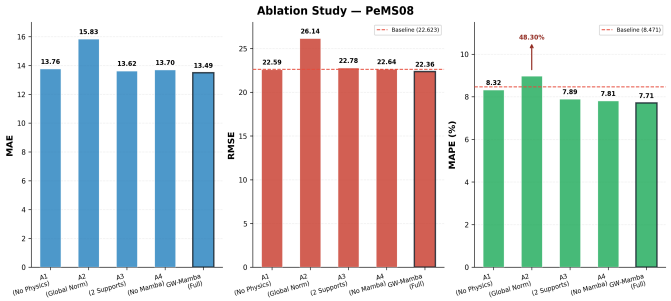


Fig. 11: Ablation study results on PeMS08 as a grouped bar chart (MAE, RMSE, MAPE per variant). The A2 bar (global normalisation) MAPE of 48.30% is shown truncated with an upward arrow. The red dashed line marks the MD-GRTN RMSE baseline (22.623). GW-Mamba (full) beats the baseline on both RMSE and MAPE.

REFERENCES

[1] S. Guo, Y. Lin, S. Li, Z. Chen, and H. Wan, "Deep spatial–temporal 3d convolutional neural networks for traffic data forecasting," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 10, pp. 3913–3926, 2019.

[2] H. Zheng, F. Lin, X. Feng, and Y. Chen, "A hybrid deep learning model with attention-based conv-lstm networks for short-term traffic flow prediction," *IEEE Transactions on Intelligent Transportation Systems*, 2020.

[3] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," *arXiv preprint arXiv:1709.04875*, 2017.

[4] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," *arXiv preprint arXiv:1707.01926*, 2017.

[5] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, "Graph wavenet for deep spatial-temporal graph modeling."

[6] J. Jiang, C. Han, W. X. Zhao, and J. Wang, "Pdformer: Propagation delay-aware dynamic long-range transformer for traffic flow prediction," *arXiv preprint arXiv:2301.07945*, 2023.

[7] H. Liu, Z. Dong, R. Jiang, J. Deng, J. Deng, Q. Chen, and X. Song, "Spatio-temporal adaptive embedding makes vanilla transformer sota for traffic forecasting," *arXiv preprint arXiv:2308.10425*, 2023.

[8] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2024.

[9] D. Li, J. Zhang, Y. Liu, J. Zhang, H. Xi, and Q. Yang, "Embedding fluid dynamics into neural networks: Toward interpretable traffic flow prediction via physics-informed learning," *IEEE Transactions on Intelligent Transportation Systems*, 2025, open Access, arXiv:2025, DOI: 10.1109/TITS.2025.11029507.

[10] Y. Wu, H. Tan, L. Qin, B. Ran, and Z. Jiang, "A hybrid deep learning based traffic flow prediction method and its understanding," *Transportation Research Part C: Emerging Technologies*, vol. 90, pp. 166–180, 2018.

[11] Y. Tian, K. Zhang, J. Li, X. Lin, and B. Yang, "Lstm-based traffic flow prediction with missing data," *Neurocomputing*, vol. 318, pp. 297–305, 2018.

[12] A. Ali, Y. Zhu, Q. Chen, J. Yu, and H. Cai, "Leveraging spatio-temporal patterns for predicting citywide traffic crowd flows using deep hybrid neural networks," in *2019 IEEE 25th international conference on parallel and distributed systems (ICPADS)*. IEEE, 2019, pp. 125–132.

[13] N. S. Chauhan and N. Kumar, "Novel confined attention-enabled hierarchical bi-lstm and bi-gru fusion: A multiscale traffic flow prediction application," *IEEE Transactions on Computational Social Systems*, vol. 11, no. 6, pp. 7541–7554, 2024.

[14] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," *Advances in neural information processing systems*, vol. 33, pp. 17 804–17 815, 2020.

[15] R. Jiang, Z. Wang, J. Yong, P. Jeph, Q. Chen, Y. Kobayashi, X. Song, S. Fukushima, and T. Suzumura, "Spatio-temporal meta-graph learning for traffic forecasting," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 37, no. 7, 2023, pp. 8078–8086.

[16] W. Kong, Z. Guo, and Y. Liu, "Spatio-temporal pivotal graph neural networks for traffic flow forecasting," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 8, 2024, pp. 8627–8635.

[17] L. Chen, L. Chen, H. Wang, and J. Zhao, "Temporal representation learning enhanced dynamic adversarial graph convolutional network for traffic flow prediction," *Scientific Reports*, vol. 15, no. 1, p. 8330, 2025.

[18] N. S. Chauhan, A. Arora, and N. Kumar, "STLTEformer: Spatio-temporal long-term embedding transformer for traffic flow prediction," *IEEE Transactions on Computational Social Systems*, 2025.

[19] Y. Bao, Q. Shen, Y. Cao, Y. Hou, W. Lu, and Q. Shi, "Multi-period diffusion generative graph recurrent transformer network for traffic flow prediction in vehicular networks," *IEEE Transactions on Vehicular Technology*, 2024.

[20] M. Hamad, M. Mabrok, and N. Zorba, "MCST-Mamba: Multivariate Mamba-based model for traffic prediction," *arXiv preprint arXiv:2507.03927*, 2025, submitted to IEEE GLOBECOM 2025, Communications Software and Multimedia track.

[21] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, and C. Zhang, "Connecting the dots: Multivariate time series forecasting with graph neural networks," in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 753–763.

[22] A. Ali, Y. Zhu, and M. Zakarya, "Exploiting dynamic spatio-temporal graph convolutional neural networks for citywide traffic flows prediction," *Neural networks*, vol. 145, pp. 233–247, 2022.

[23] G. Jin, F. Li, J. Zhang, M. Wang, and J. Huang, "Automated dilated spatio-temporal synchronous graph modeling for traffic prediction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 8, pp. 8820–8830, 2022.

[24] F. Li, H. Yan, G. Jin, Y. Liu, Y. Li, and D. Jin, "Automated spatio-temporal synchronous modeling with multiple graphs for traffic prediction," in *Proceedings of the 31st ACM international conference on information & knowledge management*, 2022, pp. 1084–1093.

[25] A. Gu, T. Dao, S. Ermon, A. Rudra, and C. Ré, "HiPPO: Recurrent memory with optimal polynomial projections," vol. 33, pp. 1474–1487, 2020.

[26] C. Song, Y. Lin, S. Guo, and H. Wan, "Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 01, 2020, pp. 914–921.

[27] P. M. Swamidass, Ed., *MAPE (mean absolute percentage error)MEAN ABSOLUTE PERCENTAGE ERROR (MAPE)*. Boston, MA: Springer US, 2000, pp. 462–462.

[28] Y. Cui, J. Xie, and K. Zheng, "Historical inertia: A neglected but powerful baseline for long sequence time-series forecasting," in *Proceedings of the 30th ACM international conference on information & knowledge management*, 2021, pp. 2965–2969.

[29] C. Shang, J. Chen, and J. Bi, "Discrete graph structure learning for forecasting multiple time series," *arXiv preprint arXiv:2101.06861*, 2021.

[30] J. Deng, X. Chen, R. Jiang, X. Song, and I. W. Tsang, "St-norm: Spatial and temporal normalization for multi-variate time series forecasting," in *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, 2021, pp. 269–278.

[31] C. Zheng, X. Fan, C. Wang, and J. Qi, "Gman: A graph multi-attention network for traffic prediction," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 01, 2020, pp. 1234–1241.

[32] Z. Shao, Z. Zhang, F. Wang, W. Wei, and Y. Xu, "Spatial-temporal identity: A simple yet effective baseline for multivariate time series forecasting," in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022, pp. 4454–4458.

Piyush is a pre-final year B.Tech student in the Department of Computer Science and Engineering at National Institute of Technology Delhi, India. His research interests include spatio-temporal deep learning, graph neural networks, and traffic flow forecasting.

Dr. Nisha Singh Chauhan received her M.Tech degree in computer science and engineering from Indian Institute of Technology, Patna and Ph.D. from Indian Institute of Technology, Roorkee (IIT-R), India. She is currently working as an assistant professor at National Institute of Technology, Delhi. Her research interests include applied deep learning and Intelligent Transportation Systems (ITS).